专业：微电子科学与工程
姓名：_ 、 _
组号：G13
日期：6 月 18 日
地点：东 4-227

浙江大学实验报告

课程名称：电子电路设计实验Ⅱ 指导老师：王子立、金向东 成绩：
实验名称：基于 STM32 的函数信号发生器的设计、组装与调试研究 实验类型：设计型

一、总体目标

1、系统框图及功能说明

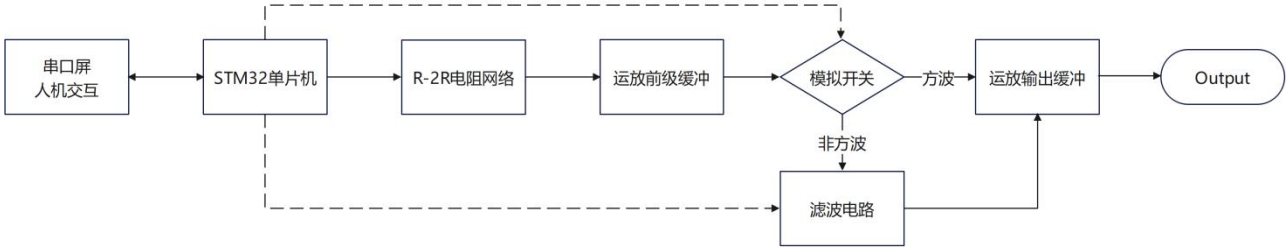


图 1 系统框图

功能说明：本系统使用串口屏与单片机进行交互，DAC 方法采用 R-2R 电阻网络，滤波部分由数字电位器组成，其中数字电位器由单片机程序控制，负责正弦波、三角波和锯齿波的滤波，方波不经过滤波直接输出，模拟开关控制不同波形信号的选择，经过运放 LM358 后输出。

二、硬件设计与调试

1、原理图设计

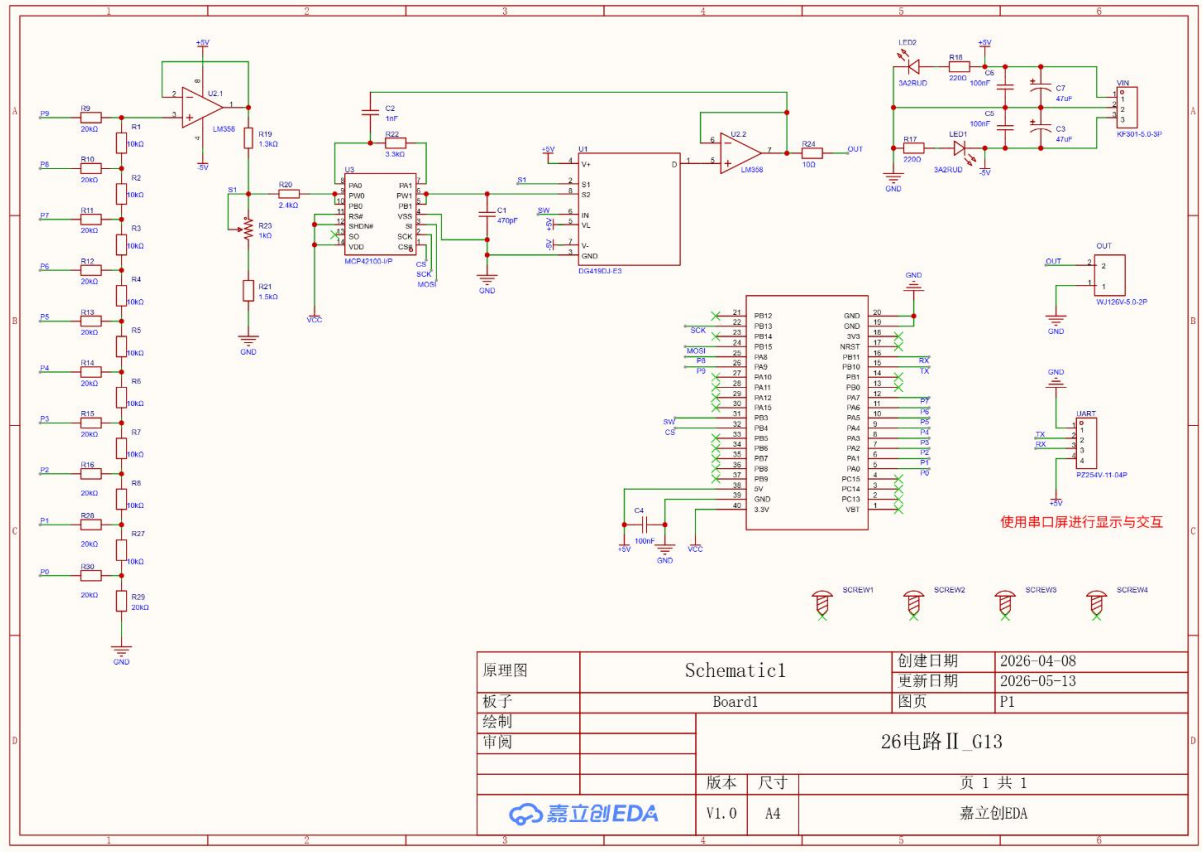


图 2 电路原理图

整体电路可分为以下几个核心模块：

①DAC 模块：由 P0-P9 组成的 10 位并行数据线驱动 R-2R 电阻网络（10kΩ/20kΩ），将单片机输出的数字量转换为模拟电压信号（如原始的阶梯波或方波）。

②运放缓冲（前级与后级）：采用 LM358 双运放。前级（U2.1）作为电压跟随器，对 R-2R 网络进行阻抗匹配和隔离；后级（U2.2）用于最终信号的输出驱动，提高带载能力。

③程控低通滤波：采用单片机经 SPI 总线控制的数字电位器（U3，MCP42100）搭配电容构成 RC 低通滤波器，可动态调节截止频率，将阶梯波平滑滤波为正弦波等平滑波形。

④模拟开关波形选择：采用 SPDT 模拟开关（U1，DG419）。受单片机 SW 引脚控制，用于在“保留高频分量的原始旁路信号”与“经过低通滤波后的平滑信号”之间进行二选一切换。

⑤人机交互：板载 UART 接口专门用于连接外部串口屏，从而实现了波形参数的实时显示与触控交互控制。

⑥电源：电路采用 +5V 和 -5V 双电源供电，确保运放和模拟开关能完整处理正负极性的交流波形。

2、PCB 设计

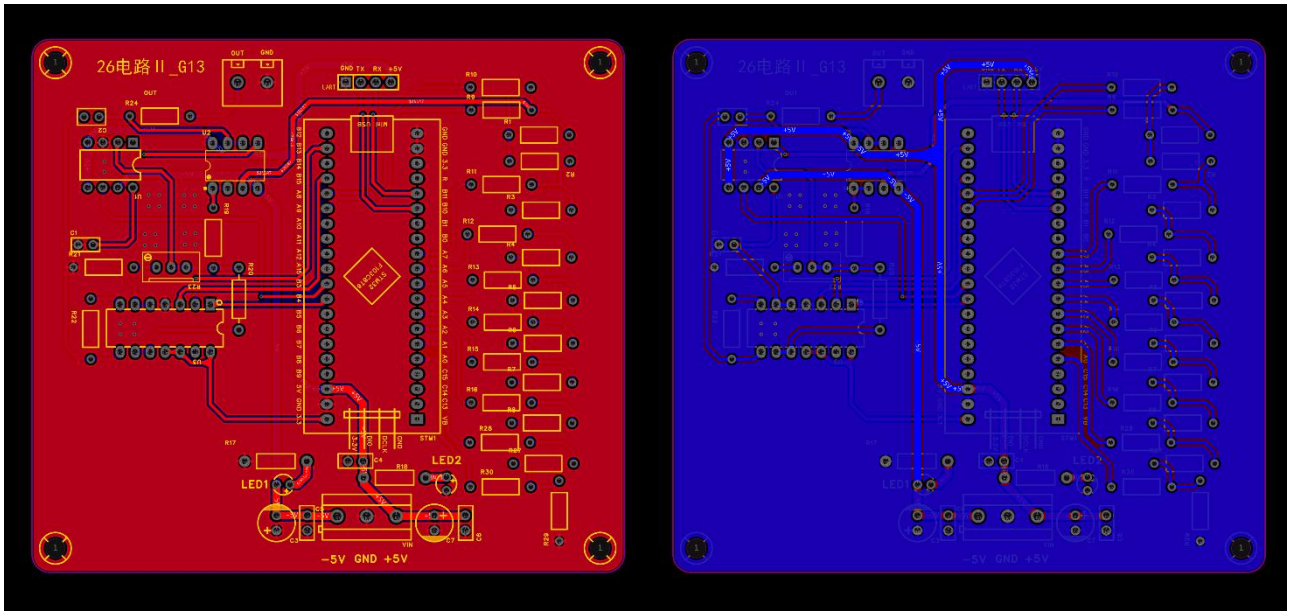


图 3　PCB 板图

本实验的 PCB 设计较好地遵循了 PCB 设计注意事项中的各项规范：

模块化布局与数模隔离方面，电路按 DAC 模块（R-2R 电阻网络）、运放缓冲级（LM358）、程控滤波器（MCP42100+RC 网络）、模拟开关（DG419）及电源电路等功能模块分区布置，STM32 数字部分与后续模拟链路相互分离，有效降低了数字噪声对模拟信号的串扰。

接插件与滤波电容放置方面，电源输入端子、UART 串口等外部接口均置于板边，便于安装接线；LM358 运放的±5V 电源引脚附近就近放置去耦电容，确保滤波效果。

布线与覆铜处理方面，信号线采用适中宽度，电源走线明显加粗以降低压降，GND 采用大面积覆铜提供低阻抗回流路径，有利于信号完整性。

工艺与校验方面，四角设置内径 3.2mm 安装孔，丝印层标注元件位号与接口标识，并在完成后执行了 DRC 校验，整体设计规范合理，为电路功能实现提供了良好的物理基础。

3、硬件调试

（1）电路仿真

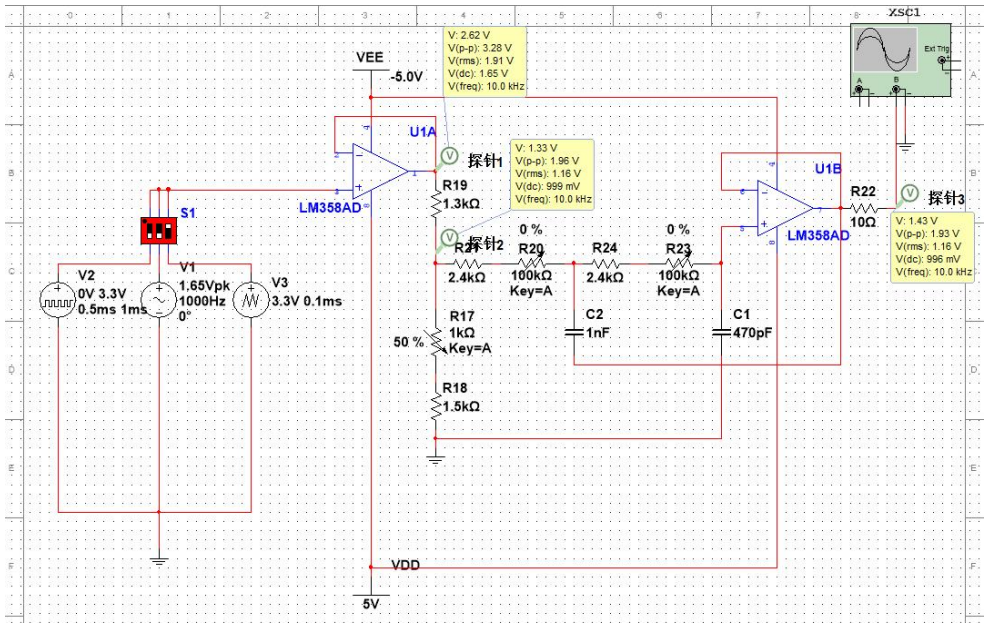


图 4　仿真电路图

仿真波形如下：

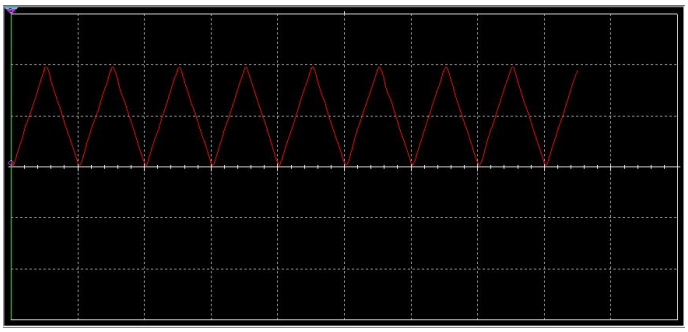


图 5　三角波仿真

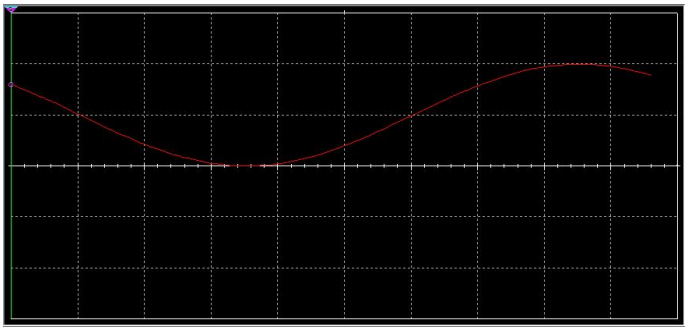


图 6　正弦波仿真

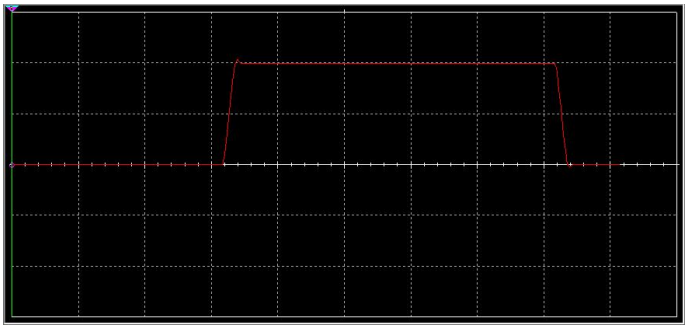


图 7　方波仿真

（2）电路焊接

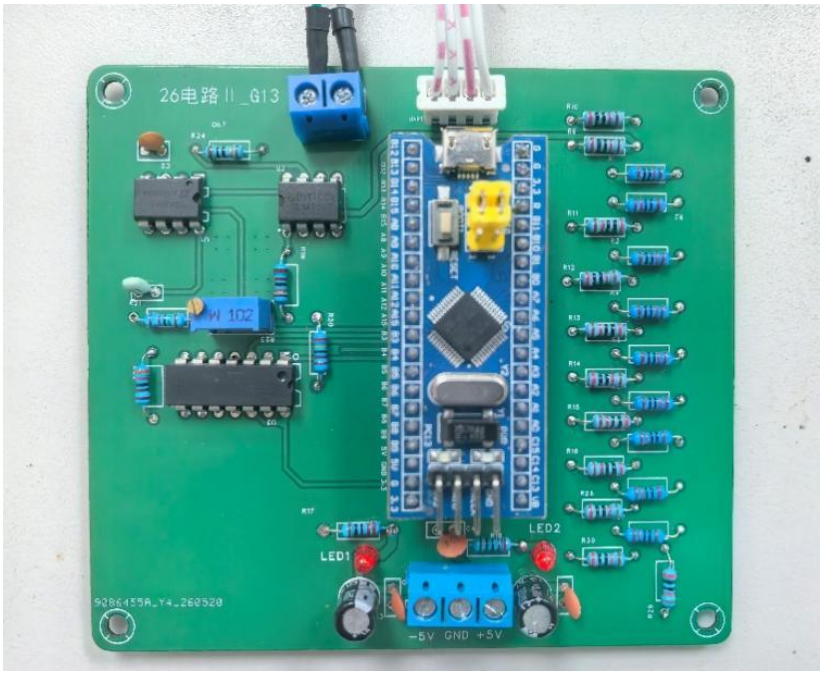


图 8　焊接成果图

经调试，电路焊接质量良好，连接情况符合设计。

（3）上电测试

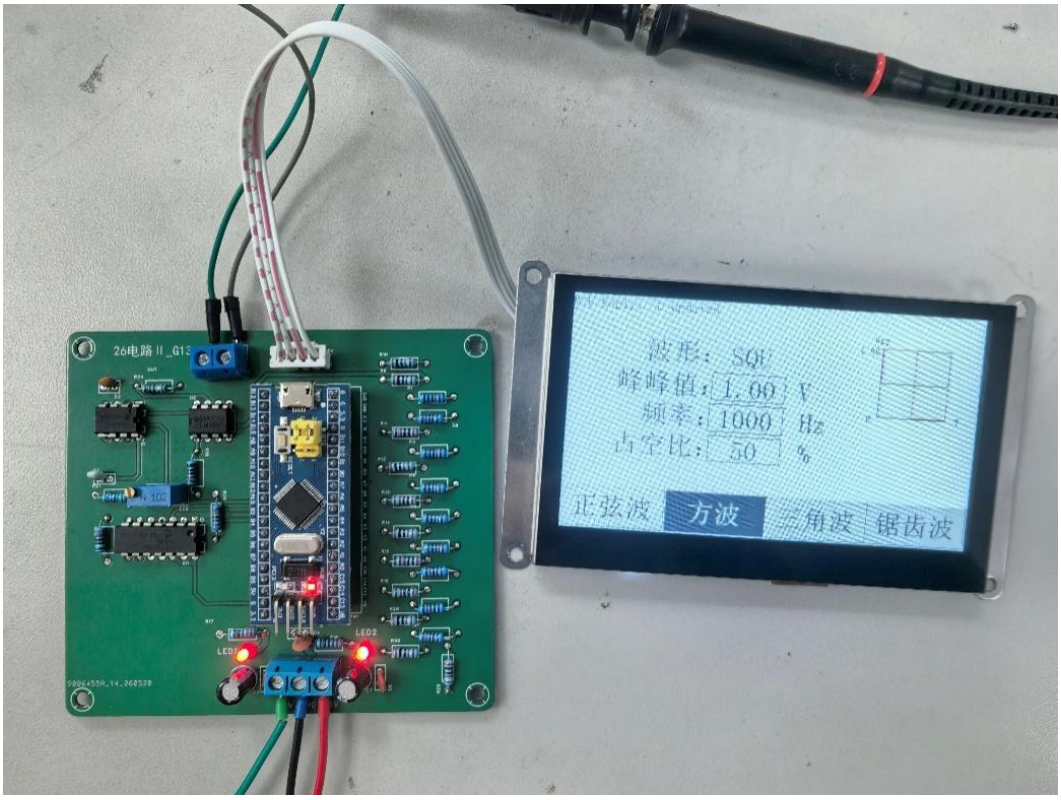


图 9　工作状态

由图 9 可知，上电后供电正常，正负 5V 电源指示 LED 灯亮起，串口屏的连接也正常，电路焊接正确。

4、硬件电路整体性能分析

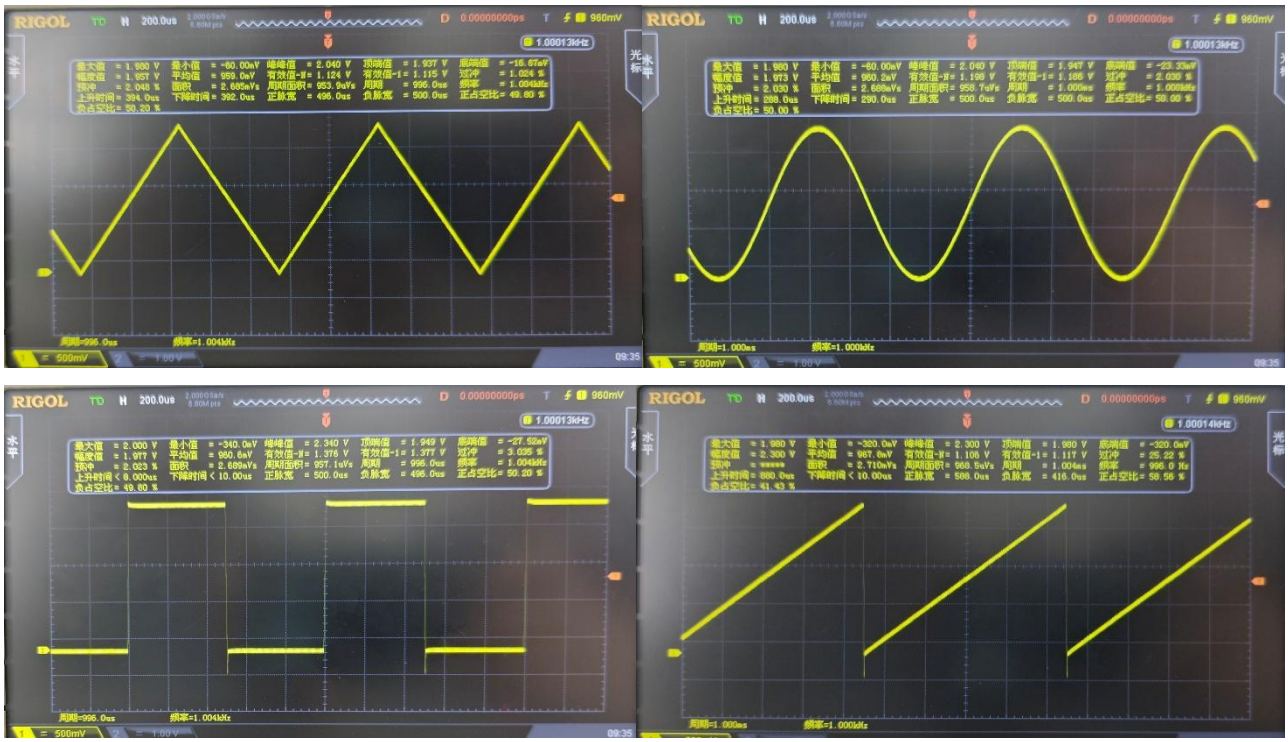


图 10　三角波、正弦波、方波与锯齿波（锯齿波为扩展内容）

设置信号峰峰值为 2V，频率为 1kHz，观察输出波形，由图 10 可知，该电路对于不同波形都能正常输出，硬件部分功能正常。

但是在实际测试中发现，方波和锯齿波的波形负半部分底部出现了一个明显的负向毛刺，而正半部分波形表现正常。该极短的尖脉冲导致示波器自动测量的峰峰值（Vpp）将其极值计入，使得在设定峰峰值 2V 的情况下测量结果比理论值高出约 0.3V，从而必须依赖光标手动对齐波形的稳态平底才能获取准确幅值。对于这种现象，可能的原因有以下 3 种：

① R-2R 电阻网络固有的“进位毛刺”现象：当方波极性切换或锯齿波复位跳变时，单片机参与转换的多个 GPIO 状态必须同时翻转。受单片机内部微小延时差异及 PCB 走线长度的影响，各引脚电平实际上无法做到绝对同步。若跳变瞬间高位引脚先关断，而低位引脚尚未完全开启，总输出电压便会在瞬间跌落至极低电位，从而在下降沿产生极快的向下尖刺。

② LM358 运放的非对称驱动特性与压摆率限制导致。LM358 属于早期低速运放，且其内部甲乙类输出级向下拉低电压的能力明显弱于向上拉电流的能力。当面对方波或锯齿波极其陡峭的下降沿时，运放无法快速吸收电流，导致瞬态响应劣化并产生负向的动态过冲，这一特性使得原本微小的数字进位毛刺被运放拉长、放大成了约 0.3V 的负向尖脉冲。

③ 数模混合信号的瞬态容性串扰导致。方波和锯齿波在发生跳变时具备极大的瞬时电压变化率（du/dt）。单片机数字控制引脚的高速下降沿，非常容易通过 PCB 板层的寄生电容，耦合到呈高阻抗状态的 R-2R 网络模拟输出端（即前级运放的同相输入端）。这种高频耦合效应会在运放的模拟输入端直接叠加出一个瞬态的负偏压尖峰。

经过光标法校准后，测得各波形情况下的输出信号峰峰值和频率如下表

	三角波	正弦波	方波（校准后）	锯齿波（校准后）	理论值
峰峰值/V	2.04	2.04	2.04	2.00	2
峰峰值相对误差	2.00%	2.00%	2.00%	0.00%	-
频率/kHz	1.004	1.000	1.004	0.996	1.000
频率相对误差	0.40%	0.00%	0.40%	-0.40%	-

从上表数据可以看出：

在峰峰值方面，除锯齿波的相对误差为 0%外，其余波形峰峰值均存在稳定的+2%的正向误差。这通常是由于前级供电基准电压略微偏高引起的系统静态误差，且可以通过调整电位器 R23 进一步减小误差，整体精度极高，符合设计要求。

在频率方面，正弦波频率零误差，三角波与方波频率微幅偏快（+0.4%），锯齿波微幅偏慢（-0.4%）。这是由于不同波形在单片机程序中查表的点数不同，进而在定时器中断处理时产生的微小指令周期截断误差，误差极小（<0.5%），表现出优异的频率稳定性。

三、软件设计与调试

1、软件工作流程框图

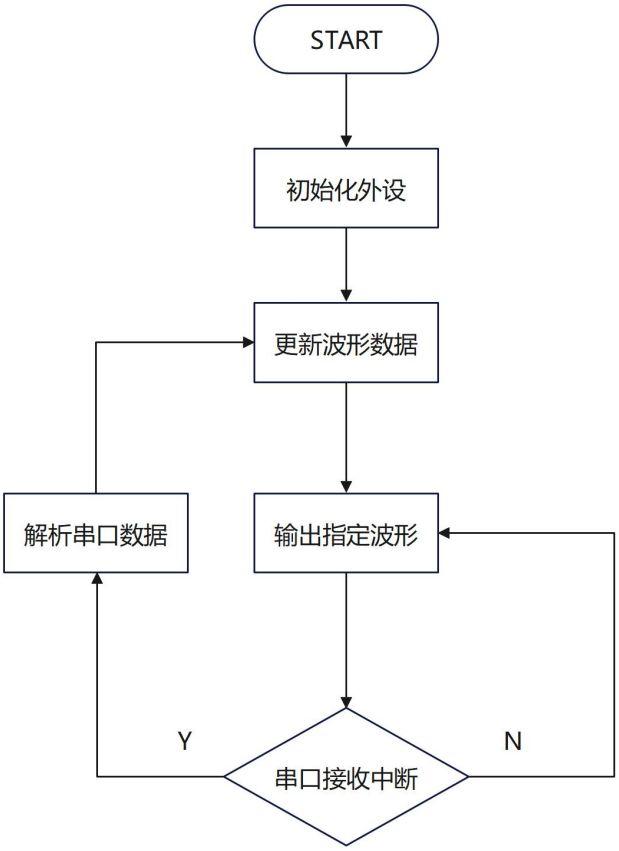


图 11(a) 总系统流程框图

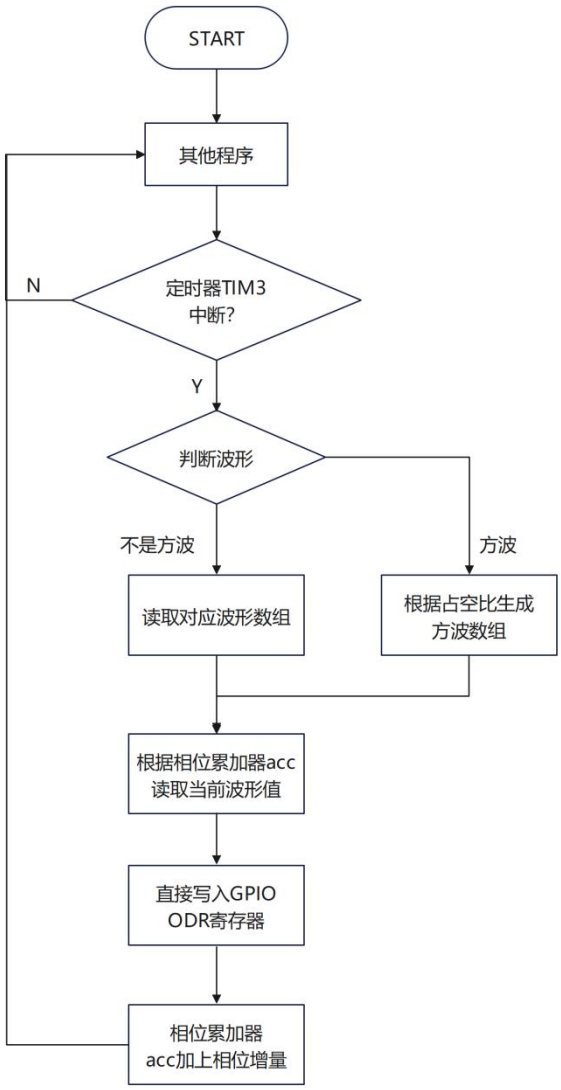


图 11(b) 波形输出部分流程框图

2、软件功能设计

（1）基础配置

①采用 STM32F103C8t6 最小系统板，使用的软件有 CubeMX、Keil5、USART HMI(串口屏设置)。

②引脚配置：

引脚名称	功能配置	对应外设	说明

PA0~PA9	GPIO_Output	R-2R 电阻网络	统一使用 GPIOA 方便直接写入寄存器
PB10	UART_Tx	串口屏 陶晶驰 T1 系列 4.3 寸	触控屏，兼顾输入输出的人机交互。同时该串口可用于调试。
PB11	UART_Rx		
PB13	SPI_SCLk	数字电位器 MCP42100	双路 100KR 数字电位器，通过 SPI 协议写入双通道阻值，用于可调滤波功能。
PB11	SPI_MOSI		
PB3	SPI_CS		
PB4	GPIO_Output	模拟开关 DG419j	使用高低电平控制通路选择，用于可调滤波

③其他配置：

定时器 TIM3 自动重装寄存器 ARR=180-1，预分频器 PSC=0，定时频率为 400KHz。开启定时器中断。

串口波特率为 115200，开启中断并启用 DMA。关闭接收过半中断。

单片机 SPI 使用 Transmit Only Master 模式。

（2）波形更新与输出功能

①波形相关变量定义：

定义波形控制结构体：

```c
typedef enum {
    WAVE_SINE     = 0,
    WAVE_SQUARE   = 1,
    WAVE_TRIANGLE = 2,
    WAVE_SAWTOOTH = 3
}WaveForm_t;

typedef struct {
    volatile WaveForm_t DAC_WaveForm;
    uint32_t            DAC_Freq;      // 目标频率 Hz
    uint16_t            Square_Duty;   // 方波占空比 0~100
    float               Vpp;           // 峰峰值电压 0~2.0V
    uint16_t            V_dig;         // 对应数字量 0~1023
} WaveData_t;
```

为方便修改，对最大幅度与 TIM3 频率进行宏定义：

```c
#define DAC_REF_FREQ 400000
#define DAC_MAX_VOLTAGE 2.0f
```

定义波形控制相关变量：

```c
//相位累加器
volatile uint32_t dds_phase_acc = 0;   //相位累加
volatile uint32_t dds_phase_inc = 0;   //相位增量

//波形控制结构体
volatile WaveData_t waveCtrl = {
    .DAC_WaveForm = WAVE_SINE,
    .DAC_Freq     = 200,
    .Square_Duty  = 50,
    .Vpp          = 2.0f,
```

```
    .V_dig        = 1023
};

uint16_t V_coeff_int = 1023;          //幅度缩放系数

//Waveform
static uint16_t sine_wave_10bit[1024] = {...};
static uint16_t square_wave_10bit[1024] = {...};
static uint16_t triangle_wave_10bit[1024] = {...};
static uint16_t sawtooth_wave_10bit[1024] = {...};
```

②相位累加器（频率调节方法）：

波形中每个点的更新频率由定时器 TIM3 定时中断确定，设定 TIM3 计时频率为$\frac{72MHz}{(acc-1)(psc-1)}$=400KHz。该频率为调试后得到的较优频率，如继续增大频率，单片机频繁进入 TIM3 中断影响系统稳定性，导致输出频率异常；如减小频率，波形更新频率降低，波形的上限频率就会相应降低。

对于频率调节方法，最初提出两个方案：一是直接调节计时器频率控制输出频率，优点是简单直观，但是受限于分频器与累加计数器的数值选择，该方案只能对应某些特定频率，对于其他频率误差较大。二是相位累加器方案，指定频率f_s，固定计时器频率f_{clk}，相位累加器设置为N位，计算出每个计时器周期增加的相位值$inc=\frac{f_s\times 2^N}{f_{clk}}$，每个周期中相位累加数$acc=acc+inc$，得到当前所需要的波形数组下标，从而输出最接近的波形值。该方案可以极好地逼近任意指定频率。本次项目采用相位累加器方案，频率更新函数如下：

```
void DDS_SetFrequency(uint32_t freq)
{
    // phase_inc = freq * 2^32 / DAC_REF_FREQ
    // phase_inc = freq * (2^32 / 200000)
    dds_phase_inc = (uint32_t)(((uint64_t)freq << 32) / DAC_REF_FREQ);
    waveCtrl.DAC_Freq = freq;
}
```

③幅度与波形输出方法：

幅度使用中间变量 uint16_t V_coeff_int = 1023 来控制。对于查表得到的数值乘以$\frac{\text{V_coeff_int}}{1024}$得到实际数值。

幅度系数更新函数如下：

```
void Coeff_Update(void)
{
    float v = waveCtrl.Vpp;
    if (v > 2.0f) v = 2.0f;
    if (v < 0.0f) v = 0.0f;
    V_coeff_int = (uint16_t)(v * 1024.0f / 2.0f);
}
```

其中将系数转化为一个无符号整型数而不是直接得到一个浮点数是因为后面查表输出均在 TIM 中断

中进行，而 STM32F103 系列无专门的 FPU，如果在中断中使用浮点运算会大大拖慢处理速度，导致频率输出异常。

当占空比更新时，使用专门的函数生成对应占空比的方波数组，以此实现占空比的自动调节，而不用借助外部数字电位器。方波生成函数如下：

```
void GenerateSquareWave(uint16_t duty_percent)
{
    if (duty_percent > 100) duty_percent = 100;

    uint16_t high_count = (uint16_t)(1024U * duty_percent / 100U);

    for (uint16_t i = 0; i < 1024; i++) {
        square_wave_10bit[i] = (i < high_count) ? 1023 : 0;
    }
    waveCtrl.Square_Duty = duty_percent;
}
```

波形输出流程如下。首先在定时器中断中调用更新函数。

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM3)
    {
        DAC_UpdateOutput();
    }
}
```

调用的更新函数会读取当前波形值，并调用输出函数：

```
void DAC_UpdateOutput(void)
{
    dds_phase_acc += dds_phase_inc;
    uint16_t index = (uint16_t)(dds_phase_acc >> 22);

    uint16_t raw_val;
    switch (waveCtrl.DAC_WaveForm)
    {
        case WAVE_SINE:     raw_val = sine_wave_10bit[index];       break;
        case WAVE_SQUARE:   raw_val = square_wave_10bit[index];     break;
        case WAVE_TRIANGLE: raw_val = triangle_wave_10bit[index];   break;
        case WAVE_SAWTOOTH: raw_val = sawtooth_wave_10bit[index];   break;
        default:            raw_val = 0;                            break;
    }

    uint16_t out_val = (uint16_t)((uint32_t)raw_val * V_coeff_int >> 10);

    WriteDACReg(out_val);
```

```
}
```

其中第四行将 acc 右移 12 位是因为波形数组只有 1024 个点，使用 acc 后 10 位即可。第 16 行解释见幅度系数更新函数处。最后直接写入输出寄存器以节省时间。将输出引脚设置为 PA0~PA9 就是为了方便这一步：

```
void WriteDACReg(uint16_t V_digital)
{
    GPIOA->ODR = V_digital & 0x03FF;
}
```

（3）串口通信功能

在本次项目中，出于方便考虑，自己定义了一套通讯代码。包头对应该讯息对应的信息类型，后面跟随具体内容，同时没有校验位。定义串口接收数组 uint8_t receive_data[50]，第一位即 receive_data[0]，以此类推。通讯表如下：

第一位	包头含义	第二位	第三位	信息含义	补充
0x0A	幅度	低位两位十六进制	高位两位十六进制	receive_data[2]*256+receive_data[1]为从串口得到的基本数值，用于传递连续可调的数值。为无符号整型 0~65535	实际为两位小数，更新波形时实际幅度为原数值/100。
0x0D	占空比				/
0x0F	频率				
0x0B	波形	0x00	/	正弦波	
		0x01	/	方波	
		0x02	/	三角波	
		0x03	/	锯齿波	

需要注意，在串口中应尽量节省时间以提高稳定性，因此定义了大量更新标志，使得具体的更新过程可以在中断外处理：

```
volatile uint8_t update_flag_f = 0; //频率更新标志
volatile uint8_t update_flag_a = 0; //幅度更新标志
volatile uint8_t update_flag_d = 0; //占空比更新标志
volatile uint8_t update_flag_b = 0; //波形更新标志
```

串口接收中断函数：

```
void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart,uint16_t size){
   if(huart == &huart3){
       if(receive_data[0]==0x0F){        //freq
              waveCtrl.DAC_Freq = receive_data[2]*256 + receive_data[1];
              update_flag_f = 1;
       }else if(receive_data[0]==0x0A){             //ampl
              waveCtrl.Vpp = receive_data[2]*256 + receive_data[1];
              update_flag_a = 1;
       }else if(receive_data[0]==0x0B){             //wavetype
              if(receive_data[1]==0x00){
                            waveCtrl.DAC_WaveForm = WAVE_SINE;
              }else if(receive_data[1]==0x01){
                            waveCtrl.DAC_WaveForm = WAVE_SQUARE;
```

```
                }else if(receive_data[1]==0x02){
                                waveCtrl.DAC_WaveForm = WAVE_TRIANGLE;
                }else if(receive_data[1]==0x03){
                                waveCtrl.DAC_WaveForm = WAVE_SAWTOOTH;
                }
        }else if(receive_data[0]==0x0D){              //duty
                waveCtrl.Square_Duty = receive_data[2]*256 + receive_data[1];
                update_flag_d = 1;
        }
        HAL_UARTEx_ReceiveToIdle_DMA(&huart3,receive_data,sizeof(receive_data));
        __HAL_DMA_DISABLE_IT(&hdma_usart3_rx,DMA_IT_HT);
      }
}
```

更加耗费时间的波形数据更新过程在 main 函数的主循环中进行

```
  while (1)
  {
        if(update_flag_f){
                DDS_SetFrequency(waveCtrl.DAC_Freq);
                if(waveCtrl.DAC_WaveForm == WAVE_SQUARE){
                        GenerateSquareWave(waveCtrl.Square_Duty);
                }
                Filter_Update(waveCtrl.DAC_WaveForm,waveCtrl.DAC_Freq);
                update_flag_f = 0;
                printf("t13.txt=\"f updated\"\xff\xff\xff");
        }else if(update_flag_a){
                waveCtrl.Vpp = waveCtrl.Vpp/100;
                Coeff_Update();
                if(waveCtrl.DAC_WaveForm == WAVE_SQUARE){
                        GenerateSquareWave(waveCtrl.Square_Duty);
                }
                Filter_Update(waveCtrl.DAC_WaveForm,waveCtrl.DAC_Freq);
                printf("t13.txt=\"A updated\"\xff\xff\xff");
                update_flag_a = 0;
        }else if(update_flag_d){
                GenerateSquareWave(waveCtrl.Square_Duty);
                update_flag_d = 0;
                Filter_Update(waveCtrl.DAC_WaveForm,waveCtrl.DAC_Freq);
                printf("t13.txt=\"D updated\"\xff\xff\xff");
        }else if(update_flag_b){
                update_flag_b = 0;
                Filter_Update(waveCtrl.DAC_WaveForm,waveCtrl.DAC_Freq);
                printf("t13.txt=\"W updated\"\xff\xff\xff");
        }
}
```

（4）可调滤波功能

首先根据 数据手册写出模拟开关 DG419 与数字电位器 MCP42100 的驱动代码，包装为两组.c 与.h 代码。根据表格所示方法进行滤波：

波形	模拟开关设置	截止频率设置	说明
方波	S0，直接输出	数字电位器电阻最小	方波由于只有高低电平，无需滤波处理阶梯状噪声。不滤波还可以保证方波波形标准。
正弦波	S1，经过滤波	5 倍原频率	正弦只需滤掉阶梯状噪声，故截止频率较低
三角波/锯齿波	S1，经过滤波	9 倍原频率	需保留自身谐波，设置截止频率较高

数字电位器阻值的计算按照电路得到为：

$$R_c = \frac{232300000}{f_c} - 3300;$$

同时引入滤波系数以较方便地调节滤波效果：

```
1 //滤波系数：1.0=>标准截止频率，增大则截止频率更高
2 volatile float filter_coeff = 3.0f;
```

（5）串口屏设置

该部分不属于单片机等实验重点内容，故简要描述。

总计设置了两个界面：主界面与数字键盘界面。按下峰峰值、频率、占空比等有方框的数字时即跳转至数字键盘界面，完成赋值后返回主界面。完成波形、峰峰值、频率、占空比等数值更改后均会按照（2）串口部分所示的方式发送数据。

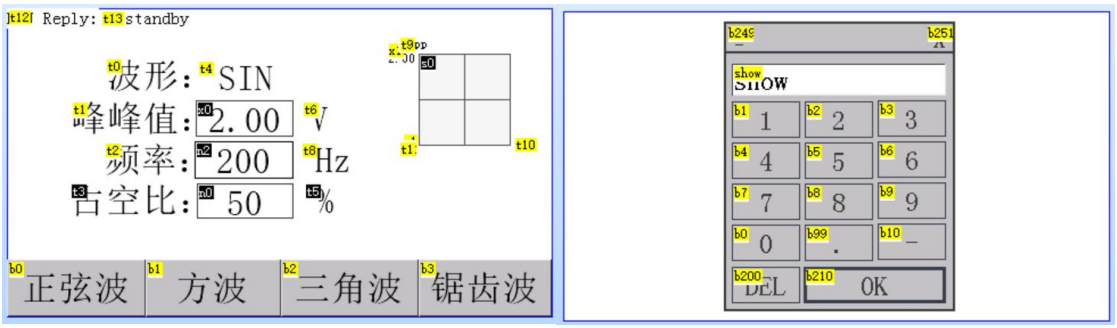


图 12(b) 串口屏主界面与数字键盘界面

同时设计了一个小波形示意图，回按照实际的波形、幅度、占空比显示波形：

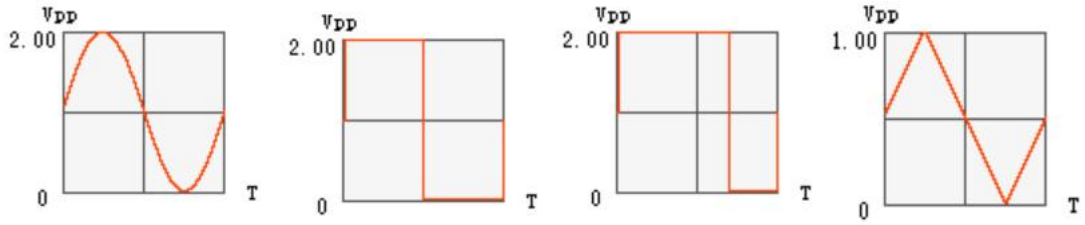


图 12(b) 波形示意图展示

Main 界面左上角的文本 t13 用于接收单片机的返回信息，方便调试。

3、软件调试：

（1）调试环境和方法

本实验的软件开发与调试基于 Keil5 MDK 集成开发环境，使用 STM32CubeMX 生成 HAL 库工程框架，编译后通过 ST-Link 仿真器下载至 STM32F103C8T6 核心板。调试方法采用"示波器观测 + 串口屏交互"双线并行的方式：硬件侧通过示波器实时观测输出波形的形态、幅值与频率；软件侧通过 UART 串口连接 USART HMI 串口触摸屏，实时下发波形类型、频率、幅值及占空比等指令并测量单片机输出与实

际波形输出，与理想结果比较并不断修正错误。

程序烧录后依次验证写 GPIOA->ODR 寄存器、DDS 相位累加器频率调节、数字电位器 SPI 驱动、模拟开关选通、与串口屏通讯等底层驱动，确认各模块正常工作后再进行联调。

（2）波形采样，串口显示

联调完成后，在串口触摸屏上分别设置不同参数组合，逐一用示波器捕获输出波形。以下按功能维度分组展示测试结果。

①频率可调功能展示

以正弦波为例（固定 2Vpp），在不同输出频率下的实测波形如下。可见在 10Hz~10kHz 宽范围内波形均稳定、平滑，DDS 相位累加算法实现了精确的频率合成。

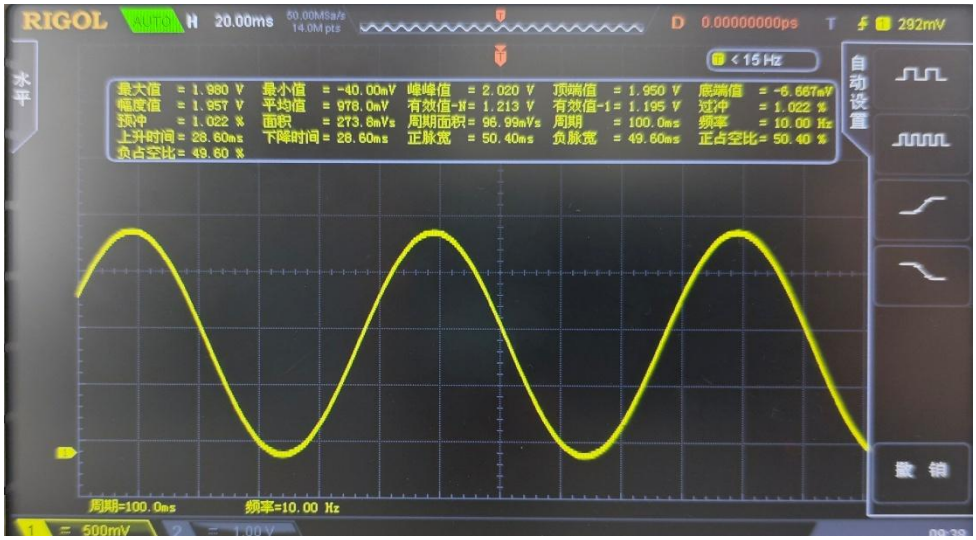


图 13(a) 2Vpp，10Hz

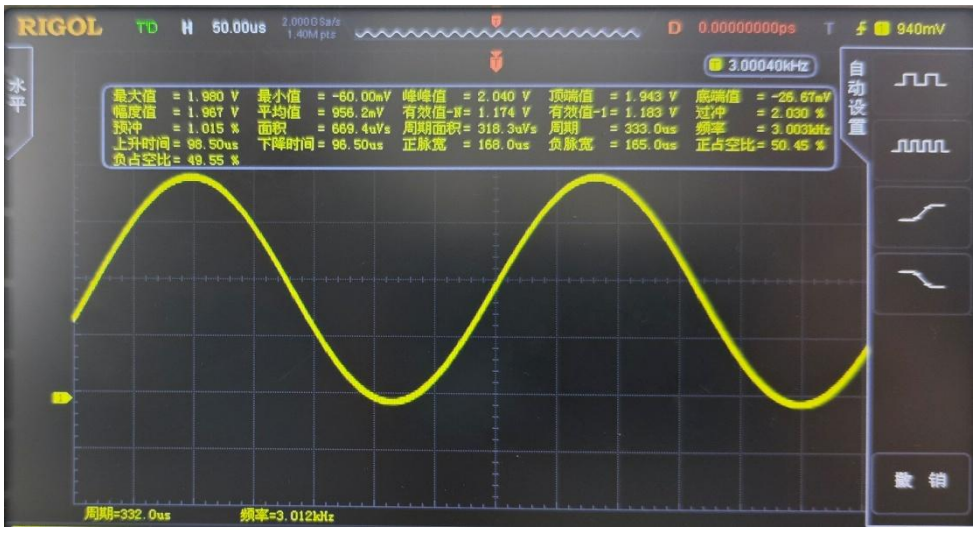


图 13(b) 2Vpp，3kHz

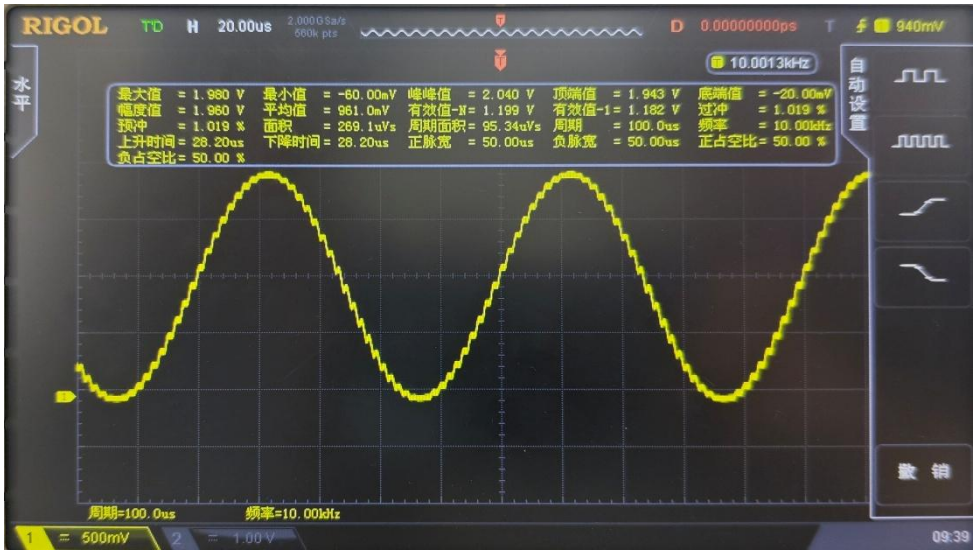


图 13(c) 2Vpp，10kHz

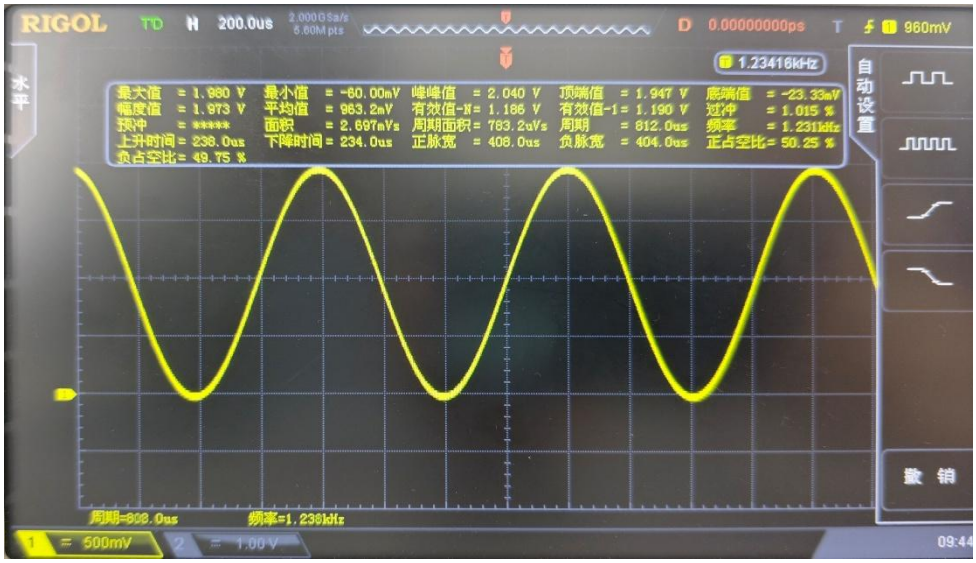


图 13(d) 2Vpp，1234Hz

对于方波和三角波，在不同的频率参数设置下，输出波形记录如下：



图 14(a) 2Vpp，10Hz



图 14(b) 2Vpp，3kHz



图 14(c) 2Vpp，10kHz

对于每种情况下的测得频率值记录如下表：

设定频率	正弦波实测	方波实测	三角波实测
10Hz	10Hz（0%）	10Hz（0%）	10Hz（0%）
1kHz	1.000kHz（0%）	1.004kHz（+0.40%）	1.004kHz（+0.40%）
3kHz	3.003kHz（+0.10%）	2.994kHz（−0.20%）	3.012kHz（+0.40%）
10kHz	10.00kHz（0%）	10.00kHz（0%）	9.960kHz（−0.40%）
1234Hz	1231Hz（−0.24%）	—	—

由上表可见，三个波形在 10Hz~10kHz 范围内的最大绝对误差均不超过 0.40%，整体频率精度优良，各波形各频点的误差均远小于 ±1%，满足实验指标要求。

观察图 13(c) 和图 14(c) 可以发现，当频率升至 10kHz 时，正弦波和三角波的波形轮廓上出现了肉眼可见的小台阶（阶梯状锯齿）。根本原因在于 R-2R DAC 的量化分辨率与每周期采样点数之间的矛盾：本系统定时器触发频率为 400kHz，当输出 10kHz 信号时，每个波形周期仅分配到 $400\text{k} / 10\text{k} = 40$ 个采样点，即 DAC 以 40 个离散台阶拼出一个完整周期。对于连续变化的波形（正弦波、三角波），40 点的量化台阶在示波器时间轴展开后清晰可辨；而方波仅在高、低两个电平间切换，本质上不依赖多级量化来构造波形轮廓，因此在同样 40 点的条件下几乎看不出台阶效应。

此处为了保证波形的幅度与形状将滤波器滤波效果调得极低，在一般的滤波下，10KHz 的波形幅度将由 2V 下降至 1V 左右，此时可以消除所有的阶梯，但三角波与锯齿波的尖角会明显变圆滑。

②幅度可调功能展示

以正弦波为例（固定 1kHz），调节输出幅值分别为 0.5Vpp、1Vpp、2Vpp，实测波形如下图所示。波形等比例缩放、基线稳定，幅值系数调节工作正常。

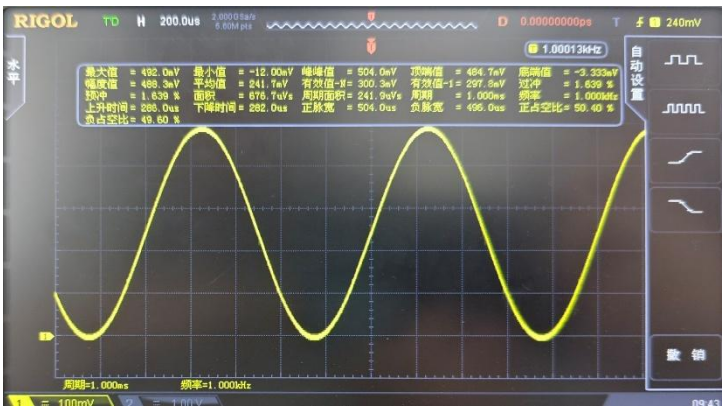


图 15(a) 0.5Vpp，1kHz

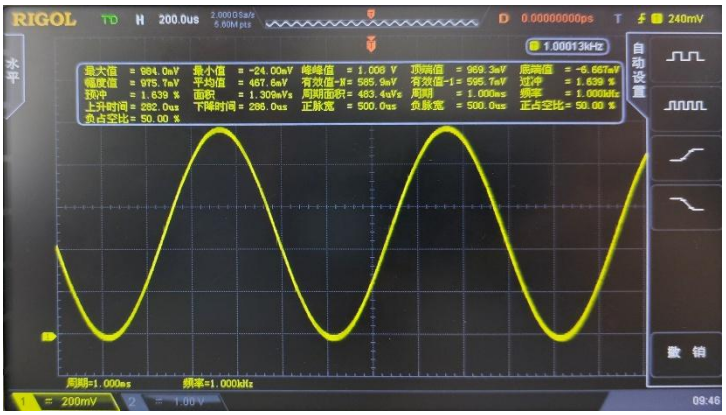


图 15(b) 1Vpp，1kHz

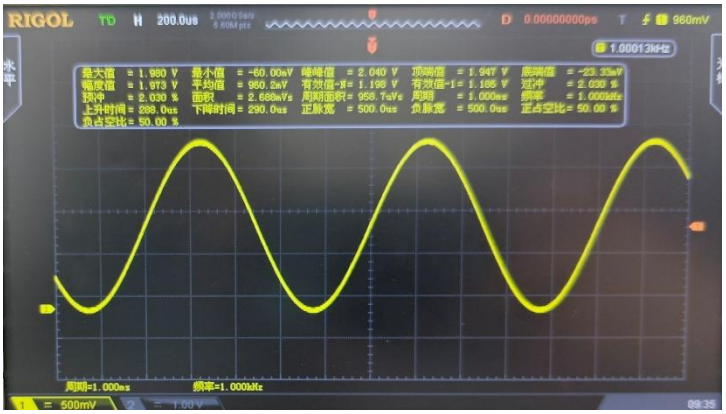


图 15(c) 2Vpp，1kHz

对于方波和三角波，也能实现 0.5V-2V 范围内的幅度可调功能，在 0.5V 的参数设置下的输出如下：



图 16 0.5Vpp，1kHz

将各设定幅值下示波器实测数据汇总如下表（方波因受负向毛刺干扰改用光标法手动量取波形的平坦段幅值）：

设定幅值	正弦波实测	方波实测（光标法）	三角波实测
0.5Vpp	0.504V（+0.80%）	0.520V（+4.00%）	0.500V（0%）
1Vpp	1.008V（+0.80%）	—	—
2Vpp	2.040V（+2.00%）	2.02V（+1.00%）	2.00V（0%）

由上表可知，测量得到的幅度值与设定值吻合良好，相对误差在 5%以内，且可以通过精细调整电位器进一步降低，总体符合设计要求。

③拓展功能：方波占空比可调

固定 2Vpp/1kHz，分别设置正占空比为 10% 与 75%，实测波形如下。高电平持续时间与设定值吻合，验证了 GenerateSquareWave()中占空比重生成算法的正确性。



图 17 2Vpp，1kHz，**占空比分别为** 10%**和** 75%

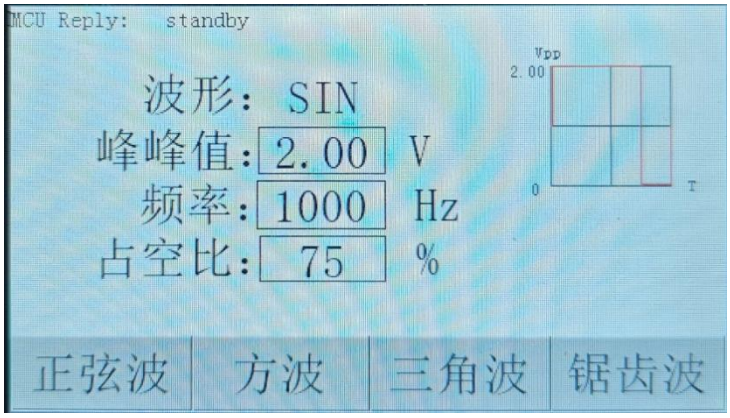


图 18 **方波类型下的串口屏操作界面**

（3）模块之间数据通讯

本系统的数据通讯链路为"串口触摸屏 → STM32 UART → 主控程序"，具体实现如下。

通讯协议设计。采用自定义帧格式：帧头 1 字节标识指令类型（0x0A 幅值、0x0B 波形、0x0D 占空比、0x0F 频率），后续字节携带对应的数值参数。例如，设置频率 1kHz 时，发送 0x0F + 频率值（两字节）；设置波形为三角波时，发送 0x0B + 0x02。协议简洁高效，无校验位，依靠帧头区分指令类别。

数据接收机制。采用 UART + DMA + 空闲中断（IDLE）方式接收不定长数据帧。由 HAL_UARTEx_ReceiveToIdle_DMA() 启动接收，当总线空闲（一帧数据发送完毕）时自动触发 HAL_UARTEx_RxEventCallback 回调函数。回调中按 receive_data[0] 的帧头判断指令类型，解析 receive_data[1] 和 receive_data[2] 组成的 16 位参数值，并置位对应的更新标志位（update_flag_f 频率 / update_flag_a 幅值 / update_flag_d 占空比 / update_flag_b 波形）。相比传统的逐字节中断接收方式，

DMA+IDLE 方案无需在中断中逐字节解析，既节省 CPU 时间，又保证了在 400kHz 定时器中断高频运行下的通讯稳定性。

主循环处理与应答。main() 的 while(1) 循环轮询上述四个更新标志位，按优先级依次处理：频率更新时调用 DDS_SetFrequency() 重算相位增量，若当前为方波则同步调用 GenerateSquareWave() 刷新占空比数组，最后调用 Filter_Update() 根据新频率重新配置数字电位器阻值以调整低通滤波截止频率；幅值更新时调用 Coeff_Update() 将电压值转换为 10 位数字系数；占空比更新时重新生成方波查找表；波形切换时更新滤波器通路选择（正弦波走 S1 滤波通道，方波走 S0 直通）。每项处理完毕后，通过 printf 向串口屏回传状态字符串（如 "f updated"），串口屏接收后将界面指定文本控件 t13 的内容刷新为对应提示文字，形成完整的人机交互闭环。

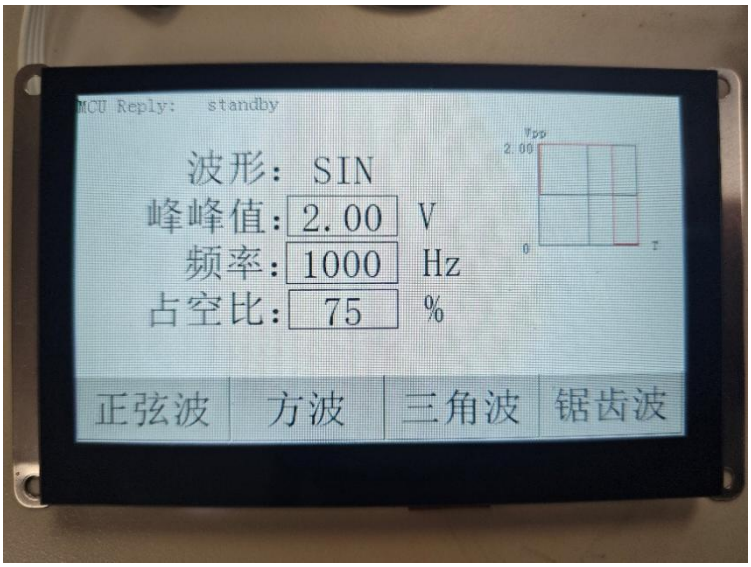


图 19 串口屏操作界面

实际调试中，触摸屏参数下发至波形响应的延迟极小（<100ms），通讯稳定无丢帧，验证了该方案的实时性和可靠性。

（4）模块性能总结：

项目	系统性能	说明
频率调节范围	10Hz~10KHz	老师验收时认为 10KHz 时波形失真程度尚可。5KHz 以下几乎无失真，使用相位累加法频率精度高。步进 1Hz。
幅度调节范围	0~2Vpp	步进 0.01V。
占空比调节范围	0~100%	步进 1%
波形种类	正弦波、三角波、方波、锯齿波	只要生成对应数组即可方便地添加新波形,拓展性良好。
串口屏人机交互	相比按钮与 LCD 屏，交互逻辑更直观高效。 频率、幅度、占空比、波形均可由串口屏直接调节	系统经验证十分稳定，延时低无通讯错误。

四、总结

1、系统装调过程中遇到的问题和解决情况

①前文硬件电路整体性能分析部分中提到的方波和锯齿波的底部负向毛刺问题：由于问题原因大概率在于

运放器件和 DAC 网络本身的性能，难以在设计层面进行解决，因此解决方案为采用光标法测量峰峰值，从而得到一个相对准确的结果。

②在调试高频滤波的过程中，重新烧录程序后，出现了大幅的幅度衰减，我们一开始以为是软件程序出错，但是回滚代码之后依然没有解决，最后经过对硬件电路的排查之后，发现问题原因在于进行烧录时连接电脑的过程中示波器探钩连接松动。虽然这是一个稀松平常的小问题，但是也警示了我们在操作过程中要随时关注电路的连接情况。

③在调试过程中，多次出现了频率偏低的情况。经检查分别是因为调试时在串口接收中断中添加了较多 printf、在定时器中断的波形更新函数中添加了浮点数运算、将定时器中断频率设置得太高。这些操作会大大增加停留在中断的时间，导致单片机无法正常执行主程序，或是无法及时进入下一次中断，大大降低了系统的稳定性。因此应该避免在中断中执行耗时长的程序，并尽量设置标志位在主程序中完成任务。

④在整个项目中，可调滤波器占用了两个集成芯片，但是却没有完全发挥出作用。在低频时点数足够滤波需求不高，在高频时又因为上限频率较低导致波形幅度衰减严重。这突出了项目开始时总体设计的重要性。最初设置的目标上限频率为 2KHz，后来提高频率时滤波器性能跟不上，严重制约了整体性能。因此在最初进行系统整体设计时，应该留有升级空间，且统筹考虑各模块的成本与性能。

2、个人的心得体会

（1）　　：

实验收获：

刚开始做仿真时，我对整个信号链路的理解还不够完整，只能一边查资料一边验证各个模块的作用。后来通过反复调整 R-2R 网络、滤波电路和运放输出部分，我逐渐明白了理论电路和实际输出之间的差距。例如本来花了大量时间调整的寄予厚望的可调滤波电路，最终却没有发挥应有的作用，可见不能盲目专注一个模块，应该注重总体设计与系统的协调。PCB 设计过程中，我也更加体会到布局、走线等的重要性，很多看似细小的处理都会影响最终波形质量。

软件部分是本次实验中耗时较多的内容。最初，因为我有电赛经验，我觉得一个 DAC 设计不会有什么难度。从 DDS 相位累加、查表输出，到串口屏通信、数字电位器控制，每个模块单独看并不复杂，但合在一起后就需要考虑协调性和稳定性，产生了许多难以解决的问题。调试中我遇到过中断函数耗时过长、频率异常、滤波效果不理想等问题。通过逐步排查和优化，我对嵌入式系统中“功能实现”和“可靠运行”之间的区别有了更深体会。这次实验让我完整经历了一次从设计到调试的过程，也让我认识到工程实践中耐心和细节同样重要。

课程评价与建议：

电子电路设计实验Ⅱ相比之前的验证性实验更接近真实的工程设计过程。它没有固定答案，也很少能一次成功，很多问题只有在仿真、画板、焊接和调试过程中才会逐渐暴露出来。通过本课程，我不仅复习和应用了模拟电路、数字电路和单片机相关知识，也学会了如何把多个模块组合成一个完整系统。

在本次函数信号发生器设计中，我们从方案选择、元件选型、原理图绘制、PCB 设计，到后期焊接、程序编写和整机调试，基本经历了电子系统开发的主要流程。这个过程让我意识到，电路设计不能只看单个模块是否可行，还要综合考虑供电、信号完整性、器件性能、软件实时性以及后期调试的方便程度。课程难度确实较高，但收获也很明显。它提高了我查阅资料、分析问题和解决问题的能力，也让我对微电子专业中“设计”和“实现”的关系有了更具体的认识。总体来说，这是一门很有挑战性、也很有实践价值的课程。

（2）　　：

实验收获：

在上这门课之前，我从来没有接触过原理图绘制、PCB 设计和单片机编程相关内容，但是通过一定的学习，我大致了解了相关软件的操作方法和元器件的选取规则。在电路设计与仿真完成后，我第一次上手进行了原理图的绘制和 PCB 设计，虽然最终的成果有较多的瑕疵，但在队友的指出和修改后，我逐渐熟

悉了电路设计的工作流程和相关操作习惯。当最终设计验收通过、电路板打印成实物送到我们手上的时候，一股小小的成就感油然而生。

接下来的焊接过程是我们在大一便接触过的内容，但是想要焊好一块板子实在不容易，有时候焊点不饱满需要补焊，而有时候一不小心又焊地过于饱满了，需要用吸锡枪吸走焊锡重新焊。这次狼狈的焊接经历真正让我认识到就算是已经掌握的技能，也需要多加练习以保持熟练度。

硬件调试通过之后，便是烧录程序进行最终的功能调试，虽然中间也出现了不少 bug，但队友凭借丰富的软件设计经验逐个解决了问题，最终我们顺利通过了验收。这次设计的经历让我完整地体验了电路设计的全流程，在学习与合作中探索未知的领域，令我收益颇丰。

课程评价与建议：

因为本学期课程的内容都是设计型的实验而不是验证型的实验，所以并不像上学期的电子电路设计Ⅰ的教学一样会有很详细的讲义，老师所能做的便是对我们的设计进行讲评和指导，因此本课程难度相比电子电路设计Ⅰ提高了很多，考虑到学生在此之前没有相关经验和知识，所以建议老师可以在课程前期多进行一些前置理论知识的讲解。

3、组员分工

（50%）：电路设计、原理图修改、PCB 设计修改、软件设计、系统调试、报告撰写

（50%）：元件选取、原理图绘制、PCB 设计、电路焊接、系统调试、报告撰写